

Day-66-Basic Excel File Handling in Python

1. Day Number

Day 66

Today you will learn the basics of working with **Excel files in Python**.

2. Topic Name

Excel Workbook and Worksheet Basics

You will learn how Python can:

- create an Excel workbook
- open an existing Excel workbook
- access a worksheet
- work with rows and columns
- read values from individual cells

We will keep everything basic today—no formulas, styling, charts, or advanced automation.

3. Connection to Yesterday

Yesterday, in **Day 65**, you worked with **CSV files**.

A CSV file stores tabular data such as:

```
Product,Price
Keyboard,1200
Mouse,600
Monitor,9000
```

Today, you will work with similar **rows and columns**, but instead of storing them in a plain-text CSV file, you will store them inside an **Excel workbook**.

Conceptually:

```
CSV
Rows + Columns
    ↓
Excel
Rows + Columns + Worksheets inside a Workbook
```

4. Important Topics

The main ideas for today are:

Term	Simple meaning
Workbook	The whole Excel file
Worksheet	One sheet inside the workbook
Row	Horizontal group of cells
Column	Vertical group of cells
Cell	One box in the worksheet
Excel library	Python package used to work with Excel files

For example:

	A	B

1	Product	Price
2	Keyboard	1200
3	Mouse	600

Here:

A1 → Product
 B1 → Price
 A2 → Keyboard
 B2 → 1200

5. Foundational Notes

What is a workbook?

A **workbook** is the entire Excel file.

For example:

products.xlsx

Think of the workbook as a notebook.

Inside the notebook, you can have several pages.

Those pages are called **worksheets**.

```
products.xlsx
|
├─ Sheet1
```

For now, you only need **one worksheet**.

What is a worksheet?

A worksheet is the familiar Excel grid containing rows and columns.

For example:

	Column A	Column B
Row 1	Product	Price
Row 2	Keyboard	1200
Row 3	Mouse	600

What is a cell?

A cell is one individual location.

Excel identifies cells using:

column letter + row number

Examples:

A1
B1
A2
B2

So:

A2 = Keyboard
B2 = 1200

6. CSV vs Excel

Both CSV and Excel can contain **rows and columns**, but they work differently.

CSV

A CSV file is basically a **plain text file**.

Product,Price
Keyboard,1200
Mouse,600

Usually the extension is:

```
.csv
```

A CSV normally represents one simple table.

Excel

An Excel file uses a special workbook format.

Usually:

```
.xlsx
```

It can contain multiple worksheets:

```
products.xlsx
├── Products
├── Customers
└── Orders
```

A simple comparison:

CSV	Excel
Plain-text format	Excel workbook format
Usually one table	Can have multiple worksheets
.csv	.xlsx
Python has built-in csv module	Usually needs an external library
Very simple data storage	Supports richer spreadsheet features

For now, think of Excel as:

A workbook containing one or more tables called worksheets.

7. Basic Excel Library

Python does not include a convenient .xlsx library in the standard library.

A beginner-friendly package commonly used for Excel files is:

```
openpyxl
```

You install it using pip.

From your terminal:

```
pip install openpyxl
```

If you are using the virtual environment you learned about on Day 61, activate the environment first.

Conceptually:

```
activate virtual environment
    ↓
pip install openpyxl
    ↓
Python project can use openpyxl
```

Then Python can import it with:

```
import openpyxl
```

You don't need to understand how openpyxl works internally. Think of it as a toolbox that gives Python commands for interacting with .xlsx files.

8. Easy Example

Imagine an Excel file called:

```
products.xlsx
```

Its worksheet contains:

A	B

Product	Price
Keyboard	1200
Mouse	600

A small piece of Python code could open that workbook:

```
import openpyxl

workbook = openpyxl.load_workbook("products.xlsx")
```

Then select the active worksheet:

```
sheet = workbook.active
```

You can access a cell:

```
sheet["A1"].value
```

This would represent:

```
Product
```

And:

```
sheet["B2"].value
```

would represent:

```
1200
```

Notice the pattern:

```
Workbook
  ↓
Worksheet
  ↓
Cell
  ↓
Value
```

This is an important mental model for today's lesson.

9. Problem Statement

Create or open a very small Excel workbook containing **product names and prices**.

Your worksheet might look like:

Product	Price
Keyboard	1200
Mouse	600
Monitor	9000

Your Python program should:

1. Work with an `.xlsx` workbook.
2. Access the worksheet.
3. Read a few cells.
4. Print some product names and prices.

Keep the workbook very small.

You do **not** need formulas, formatting, colors, charts, or complicated Excel operations.

10. Concepts Used

This exercise practices several concepts you already know alongside a few new ones.

Python concepts

```
variables  
print()  
imports  
file paths  
basic conditions
```

New Excel concepts

```
workbook  
worksheet  
row  
column  
cell  
cell value
```

And the external package:

```
openpyxl
```

11. Thought Process

Before writing code, think about the problem in layers.

Step 1 — What file am I working with?

You need an Excel file such as:

```
products.xlsx
```

Step 2 — What is inside the file?

A workbook contains a worksheet.

```
products.xlsx  
  ↓  
worksheet
```

Step 3 — What is inside the worksheet?

Rows, columns, and cells.

```
worksheet
  ↓
cells
```

Step 4 — Which cells contain the data?

For example:

```
A1 → Product
B1 → Price

A2 → Keyboard
B2 → 1200
```

Step 5 — What value do I actually want?

You don't normally want the cell object itself.

You want the **value stored inside the cell**.

Conceptually:

```
cell
  ↓
.value
  ↓
Keyboard
```

Step 6 — What should the program display?

Something simple such as:

```
Keyboard - 1200
Mouse - 600
Monitor - 9000
```

12. Beginner-Friendly Pseudocode

Do not worry about exact Python syntax yet.

```
START

import the Excel library

open the Excel workbook
```

```
select the worksheet

read the product name from a cell
read the price from another cell

print the product and price

read another product
read its price

print them

END
```

If you decide to **create** the workbook instead:

```
START

import the Excel library

create a new workbook

select the worksheet

put "Product" in the first header cell
put "Price" in the second header cell

put a product name in the next row
put its price beside it

save the workbook

read a few cell values

print the values

END
```

13. Suggested Solving Approach

Use a **simple file-handling approach**.

Think of it similarly to the text-file and CSV exercises you already completed:

```
choose file
  ↓
open/load file
```

```
↓  
access data  
↓  
read values  
↓  
print values
```

For Excel, one extra layer appears:

```
Excel file  
↓  
Workbook  
↓  
Worksheet  
↓  
Cell  
↓  
Value
```

Start by reading individual cells rather than trying to process a large spreadsheet.

For example, first understand:

```
sheet["A2"].value
```

before worrying about loops over many rows.

14. Easy Edge Cases

Empty worksheet

Suppose the worksheet exists but contains no product data.

You could encounter:

```
A1 → empty  
A2 → empty  
B2 → empty
```

Python may represent an empty cell's value as:

```
None
```

Conceptually:

```
if cell contains no value:  
    handle it as empty
```

You don't need complicated error handling yet.

Missing cell value

Imagine:

Product	Price
Keyboard	1200
Mouse	
Monitor	9000

The mouse row has no price.

Conceptually:

```
product = Mouse
price = None
```

Your program should avoid assuming every cell always contains data.

You might eventually think:

```
if price exists:
    print product and price
else:
    print that the price is missing
```

That's enough edge-case thinking for today.

15. Expected Input and Output

Expected Excel file

```
products.xlsx
```

Worksheet:

+-----+	+-----+	
Product	Price	
+-----+	+-----+	
Keyboard	1200	
Mouse	600	
Monitor	9000	
+-----+	+-----+	

Possible expected terminal output

```
Keyboard - 1200  
Mouse - 600  
Monitor - 9000
```

Your exact formatting can be slightly different.

The important goal is that Python successfully obtains the values from the worksheet.

16. Hint Only

Here are the main pieces you will probably need:

```
import openpyxl
```

To open an existing workbook, investigate:

```
openpyxl.load_workbook(...)
```

To access its default/current worksheet, investigate:

```
workbook.active
```

A cell can be accessed using a reference such as:

```
sheet["A2"]
```

And remember:

```
cell object ≠ value inside the cell
```

Look for the property that gives you the **value** stored in that cell.

For creating a new workbook, investigate:

```
openpyxl.Workbook()
```

And after creating or changing a workbook, remember that it must eventually be **saved to an .xlsx file**.

Today's core mental model

```
products.xlsx  
  ↓  
Workbook  
  ↓
```

```
Worksheet
  ↓
Row / Column
  ↓
Cell
  ↓
Value
```

Once this hierarchy makes sense, basic Excel handling in Python becomes much easier.